Military College of Signals
National University of Science and Technology (NUST)
Department of Computer Software Engineering

PBL – PHASE 3

Course Code: SE-314

Course Title: Software Construction

Submitted By: 1. Hamza Haider (454989)
2. Hassnain Bin Nisar (456079)
3. Maaz Ahmad (469539)
4. Ahtisham Arif (460560)
5. GC Zaeem Sajid (486850)
6. GC Tayyab (486856)

Submitted To: Lec Fawad Khan

Course: BESE 29-A

Date: 4th Dec, 25

Contents

1. Core Features & Implementation 3

1.1 Multi-User Account Management 3

1.2. Personalized Dashboard & Visualization 5

1.3. Dynamic Location Management 7

1.4. Intelligent AI Health Assistant (Chatbot) 8

1.5. Proactive Alerting System 9

1.6. Historical Data Analysis 12

1.7. Administrator Control Panel 13

2. Installation & User Manual 14

2.1 System Prerequisites 14

2.2 Configuration Setup 15

2.4 Running the System (IDE Method) 15

2.5 Running the System (Command Line / Production) 15

3. System Validation & Testing Strategy 16

3.1. Backend Validation (Integration & Unit Testing) 16

3.2. Frontend Validation (Component Testing) 17

3.3. Validation Conclusion 18

4. Deployment & Configuration 18

4.1. Backend Deployment (Railway) 18

4.2. Frontend Deployment (Vercel) 20

4.3. Production URL 21

1. Core Features & Implementation

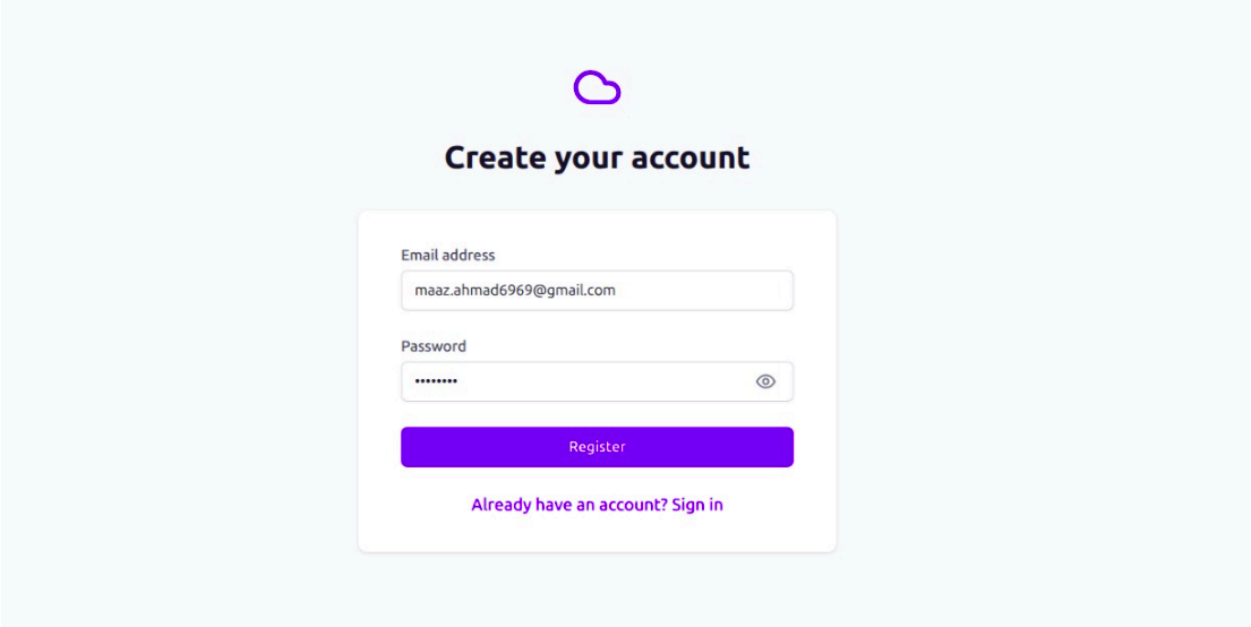
This section details the five primary features of the Intelligent Air Quality Monitoring System, demonstrating the user journey from registration to data analysis.

1.1 Multi-User Account Management

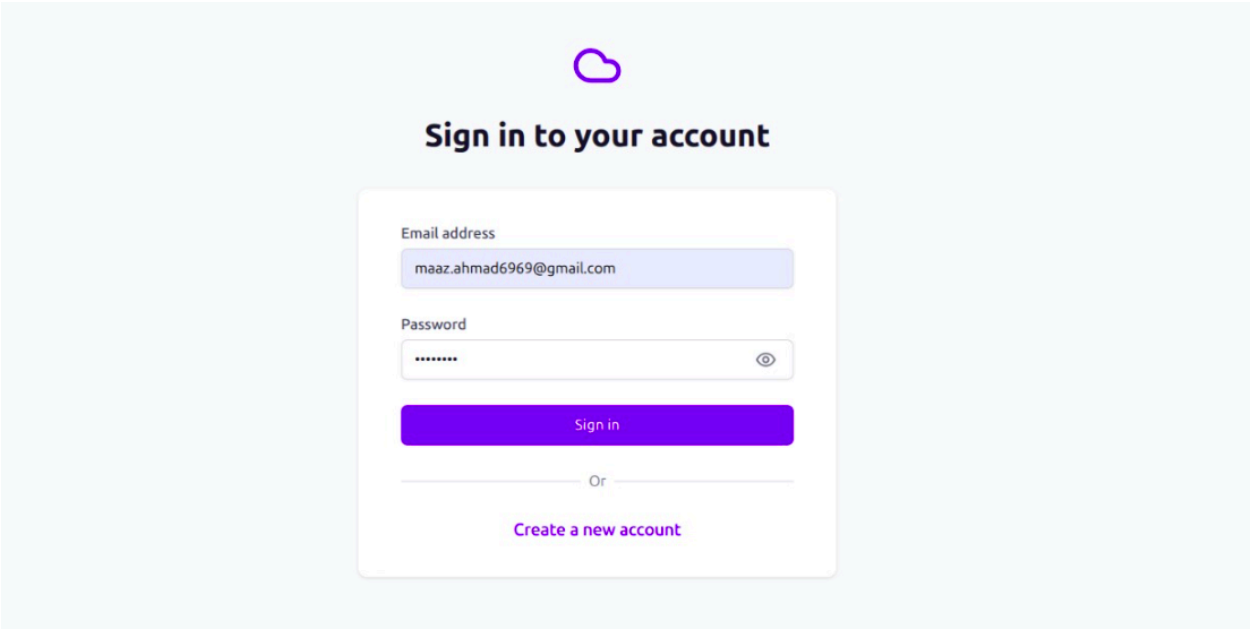
The system supports a secure, multi-user environment where each user maintains an isolated profile and personalized settings.

- **Registration:** New users can sign up securely. The system validates email formats and password strength on the client side before sending a request to the backend. Upon successful registration, a JWT token is issued for session management.
- **Authentication:** Returning users log in via a secure form. The system uses stateless JWT authentication, ensuring scalability.
- **Profile Isolation:** Each user's data (home city preference, alert history) is stored independently in the PostgreSQL database, ensuring privacy and personalization.

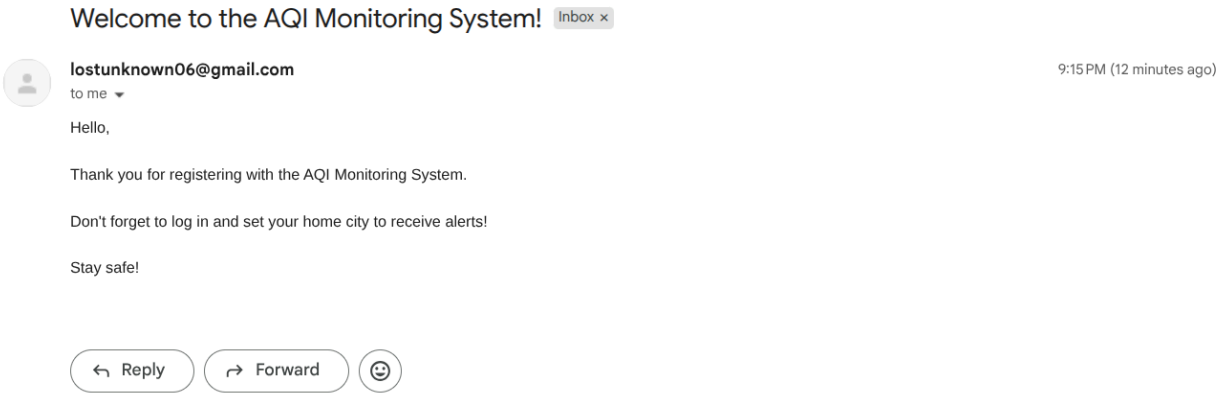
Registration



Sign in

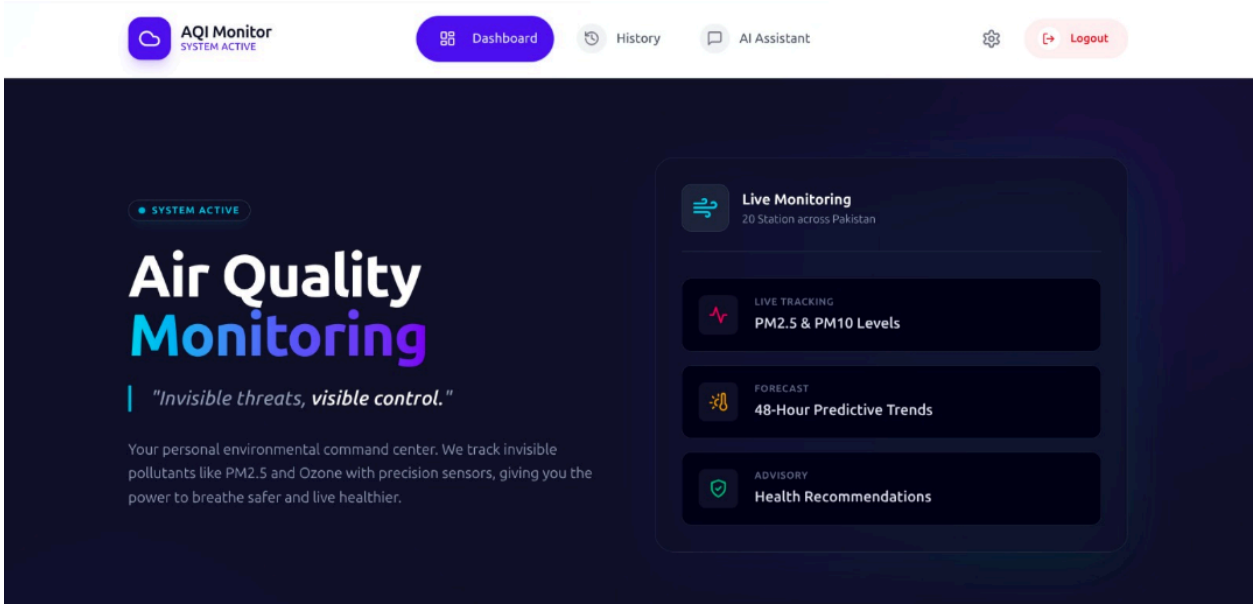


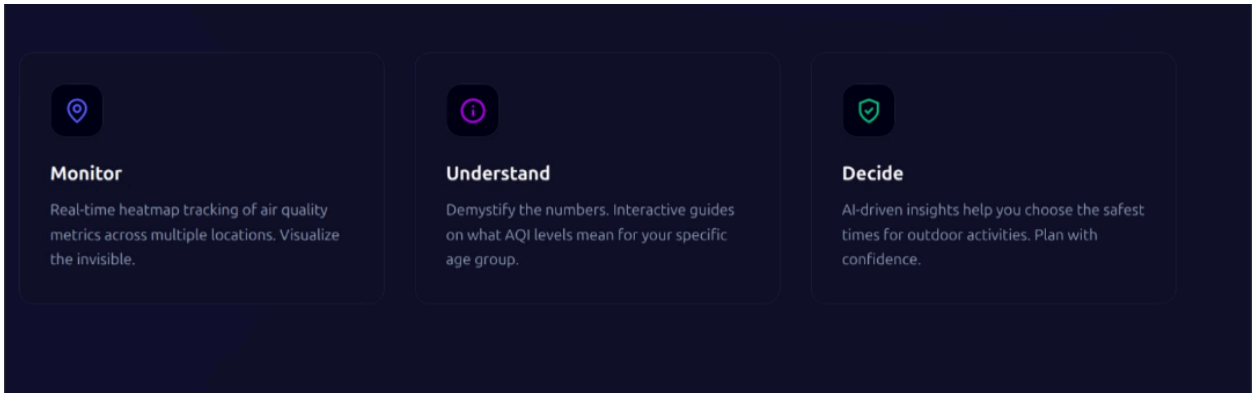
Email Notification



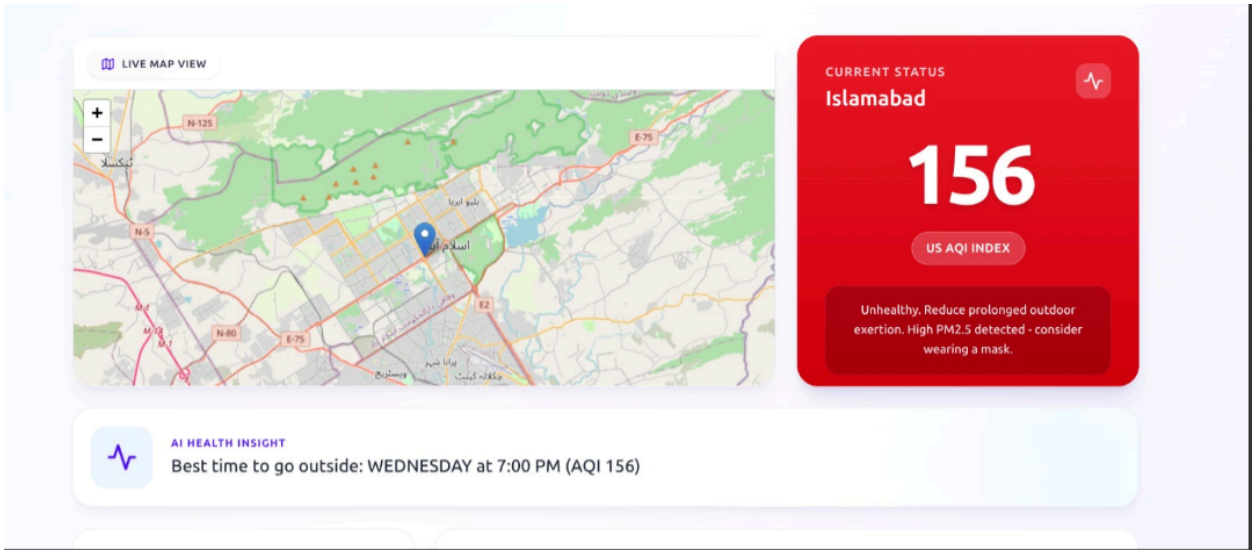
1.2. Personalized Dashboard & Visualization

The Dashboard is the central hub of the application, designed to transform complex environmental data into immediate, actionable insights.

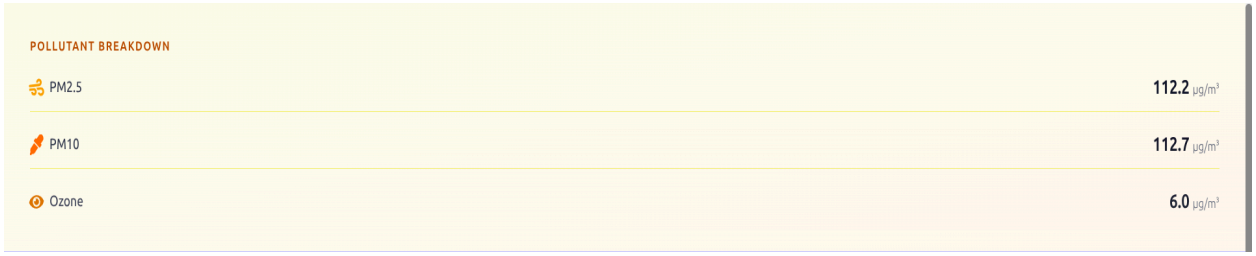




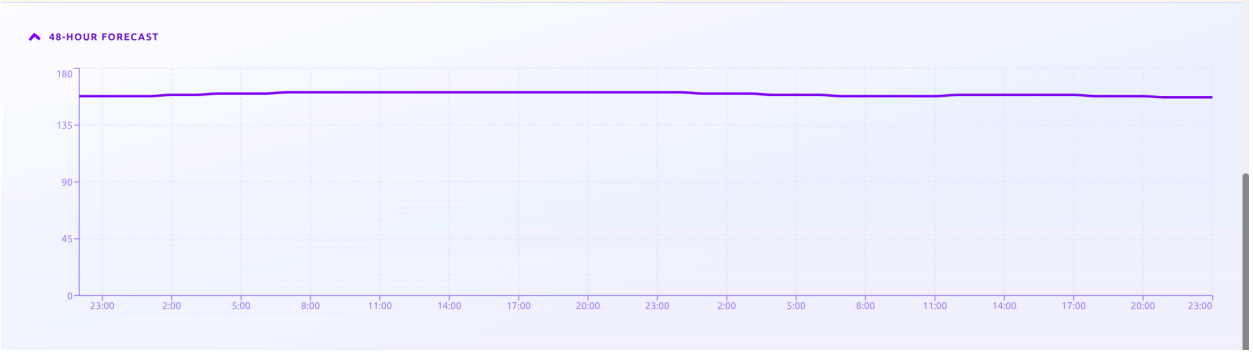
- **Real-Time AQI Display:** The current Air Quality Index is displayed prominently in a color-coded card (Green for Good, Maroon for Hazardous), allowing users to assess risk at a glance.



- **Granular Pollutant Breakdown:** Beyond the summary AQI, the dashboard provides a scientific breakdown of specific pollutants:
    - **PM2.5:** Fine particulate matter (the primary smog component).
    - **PM10:** Coarse dust particles.
    - **Ozone (O_3):** Ground-level ozone levels.



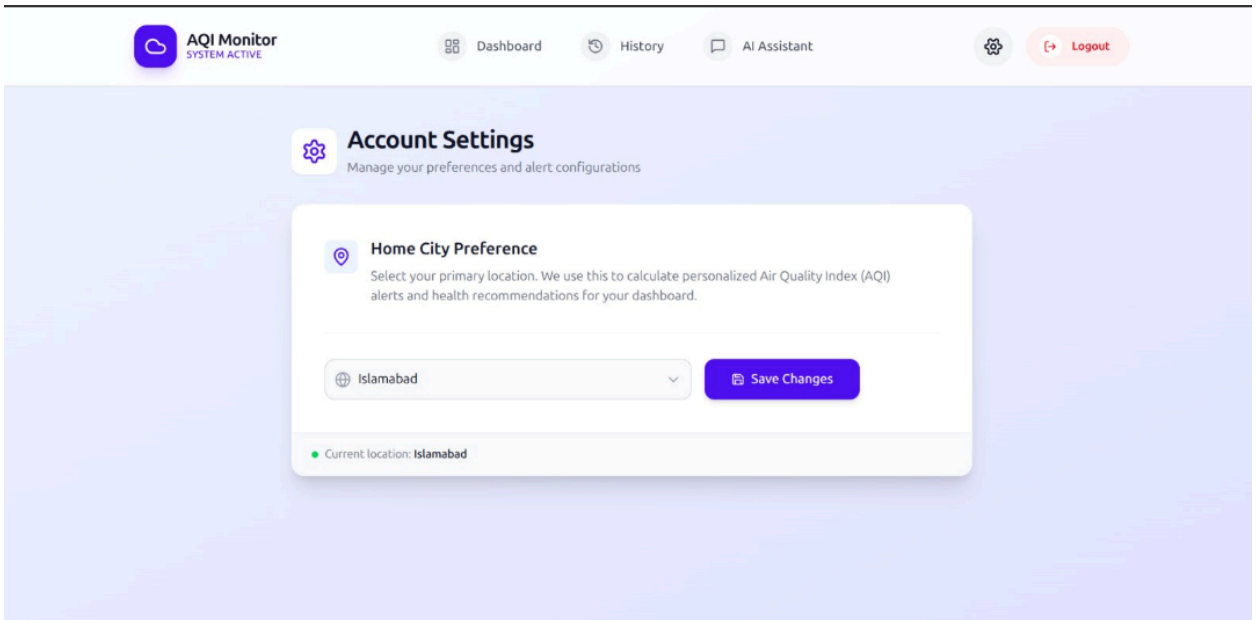
- **48-Hour Forecast:** A line chart visualizes the predicted air quality trend for the next two days, powered by the Open-Meteo forecast model, enabling users to plan outdoor activities.



1.3. Dynamic Location Management

The system is not static; it adapts to the user's location. Users can update their "Home City" at any time, instantly recalibrating the entire dashboard and alerting engine.

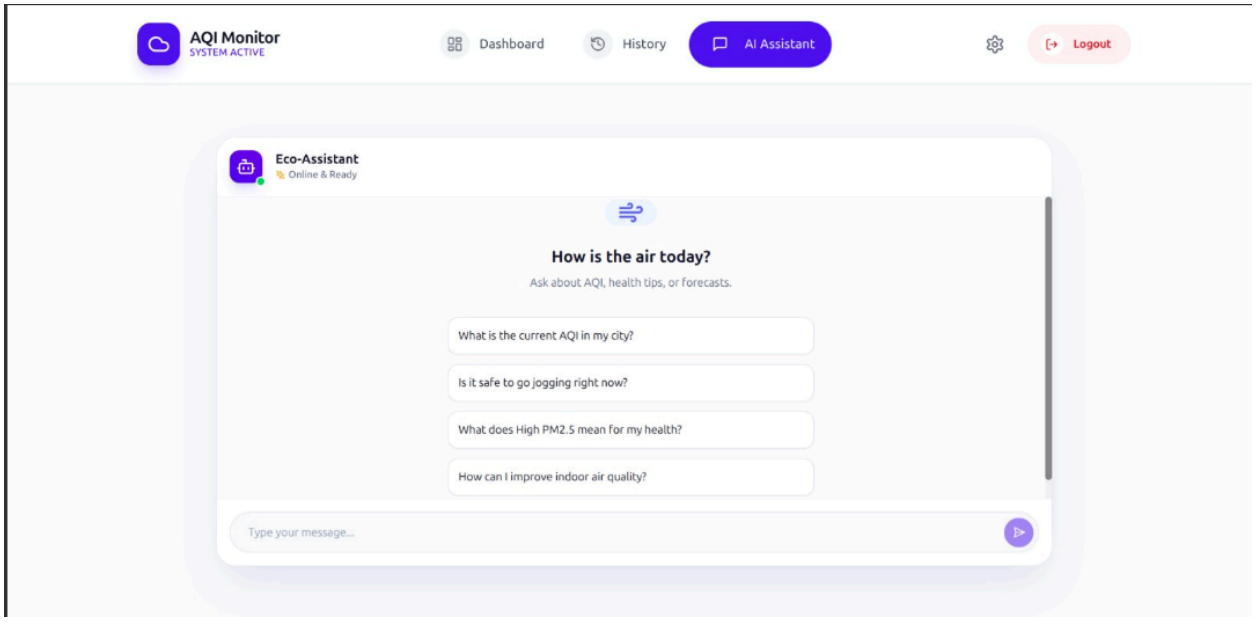
- **City Selection:** A dedicated Settings page allows users to select from a supported list of 20 major Pakistani cities (e.g., Lahore, Karachi, Islamabad).
- **Immediate Feedback:** Upon selection, the backend updates the user's profile in the database, and the frontend React Query cache is invalidated to immediately fetch fresh data for the new location.

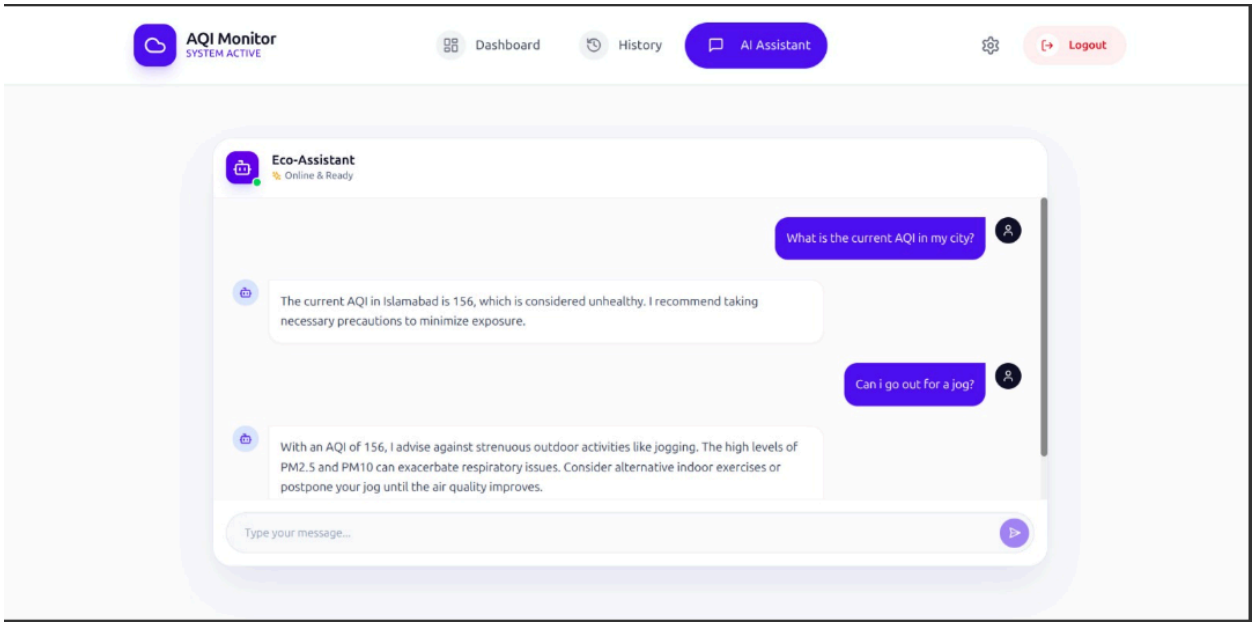


1.4. Intelligent AI Health Assistant (Chatbot)

This feature differentiates the system from standard monitoring tools by providing context-aware medical advice via a Generative AI integration (Groq Llama-3).

- **Context Injection (RAG):** When a user asks a question (e.g., "Can I go for a run?"), the backend secretly injects the user's *current* city and *live* AQI data into the AI prompt.
- **Personalized Advice:** The AI responds not with generic text, but with specific advice based on the *actual* pollution levels (e.g., "Since the AQI in Lahore is 250, outdoor running is dangerous. Stay indoors.").
- **Guardrails:** The AI is instructed to act as a serious health assistant and strictly refuses to hallucinate data for unsupported cities.





1.5. Proactive Alerting System

The system moves beyond passive monitoring to active protection through an automated notification engine.

- **Automated Polling:** The backend AqiDataPoller runs every 15 minutes to check air quality across all monitored cities.

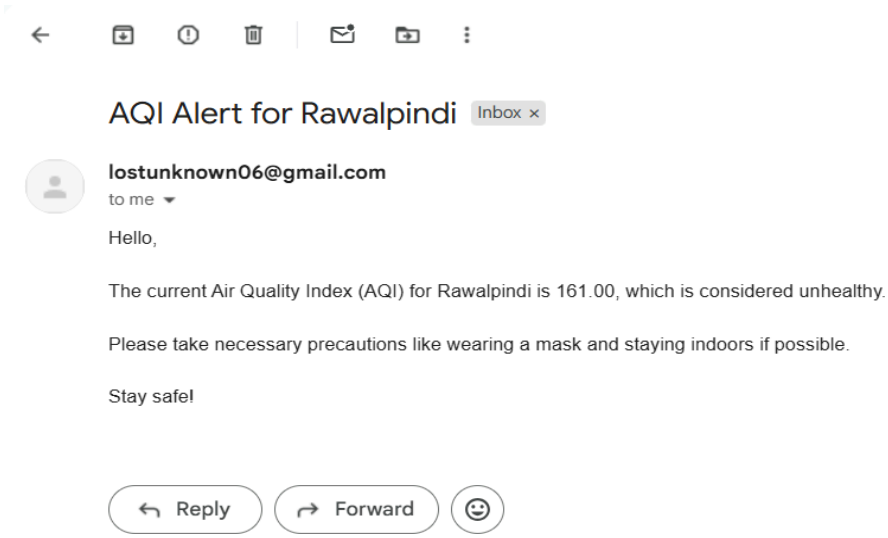
```
--- [AQI Poller] Polling Detailed Data ---
Saved Karachi: AQI=91
Saved Lahore: AQI=166
[AQI Poller] Sent 0 alerts for Lahore
Saved Faisalabad: AQI=179
[AQI Poller] Sent 0 alerts for Faisalabad
Saved Rawalpindi: AQI=156
```

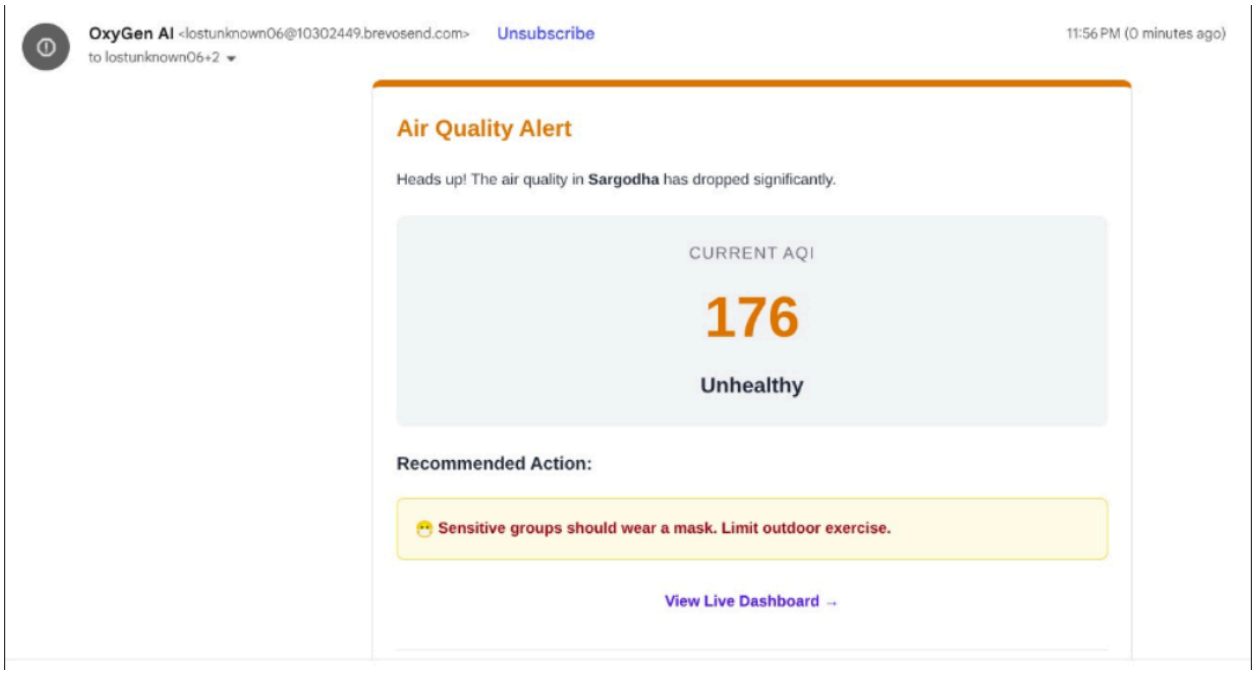
- **Threshold Trigger:** If a city's AQI exceeds the "Unhealthy" threshold (150), the system identifies all users subscribed to that city.

```
[AQI Poller] Sent 2 alerts for Rawalpindi
```

- **Email Dispatch:** An alert email is sent via Gmail SMTP, warning the user of hazardous conditions.

```
Date: Mon, 1 Dec 2025 21:18:04 +0500 (PKT)
To: maaz.ahmad.work@gmail.com
Message-ID: <1844169598.6.1764605884184@maazahmad-HP-EliteBook-840-G6>
Subject: AQI Alert for Islamabad
MIME-Version: 1.0
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

Hello,

The current Air Quality Index (AQI) for Islamabad is 156.00, which is considered unhealthy.

Please take necessary precautions like wearing a mask and staying indoors if possible.

Stay safe!
.
250 2.0.0 OK  1764605885 ffacd0b85a97d-42e1c5d618csm26717137f8f.14 - gsmtp
DEBUG SMTP: message successfully delivered to mail server
QUIT
```





- **Spam Prevention:** A "Throttling" mechanism ensures users receive a maximum of one alert per 24 hours per city, preventing inbox flooding.

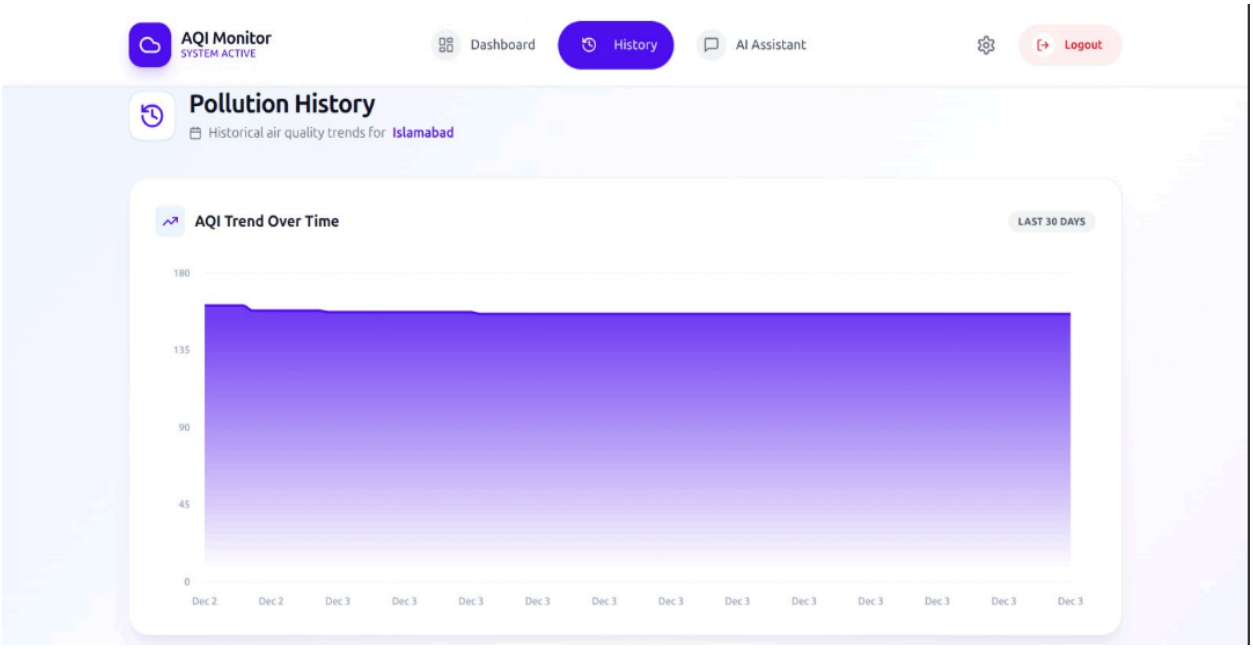
  We already sent for Islamabad and Rawalpindi 15 minutes ago, so it don't send again

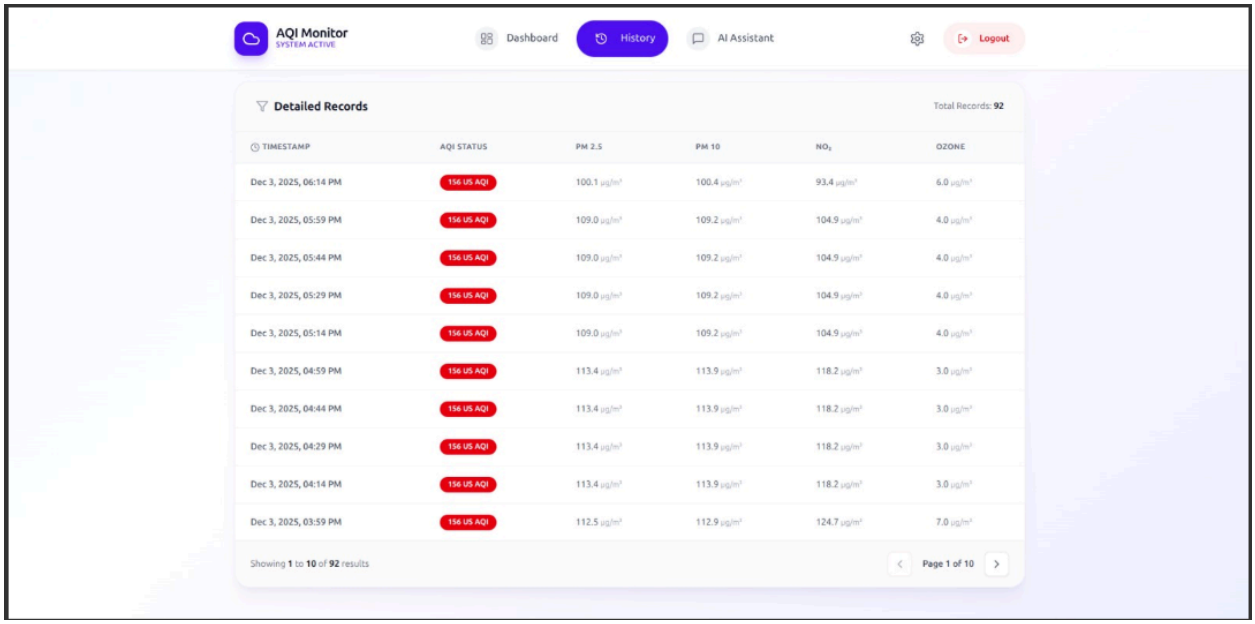
```
Saved Lahore: AQI=166
[AQI Poller] Sent 0 alerts for Lahore
Saved Faisalabad: AQI=179
[AQI Poller] Sent 0 alerts for Faisalabad
Saved Rawalpindi: AQI=156
[AQI Poller] Sent 0 alerts for Rawalpindi
```

1.6. Historical Data Analysis

To support long-term health tracking, the system maintains a history of air quality readings.**Trend Visualization:** The History page features an **Area Chart** that plots AQI fluctuations over time, helping users identify patterns (e.g., smog spikes in the evening).

- **Detailed Data Table:** A tabular view provides raw access to historical timestamps and pollutant concentrations for deeper analysis.

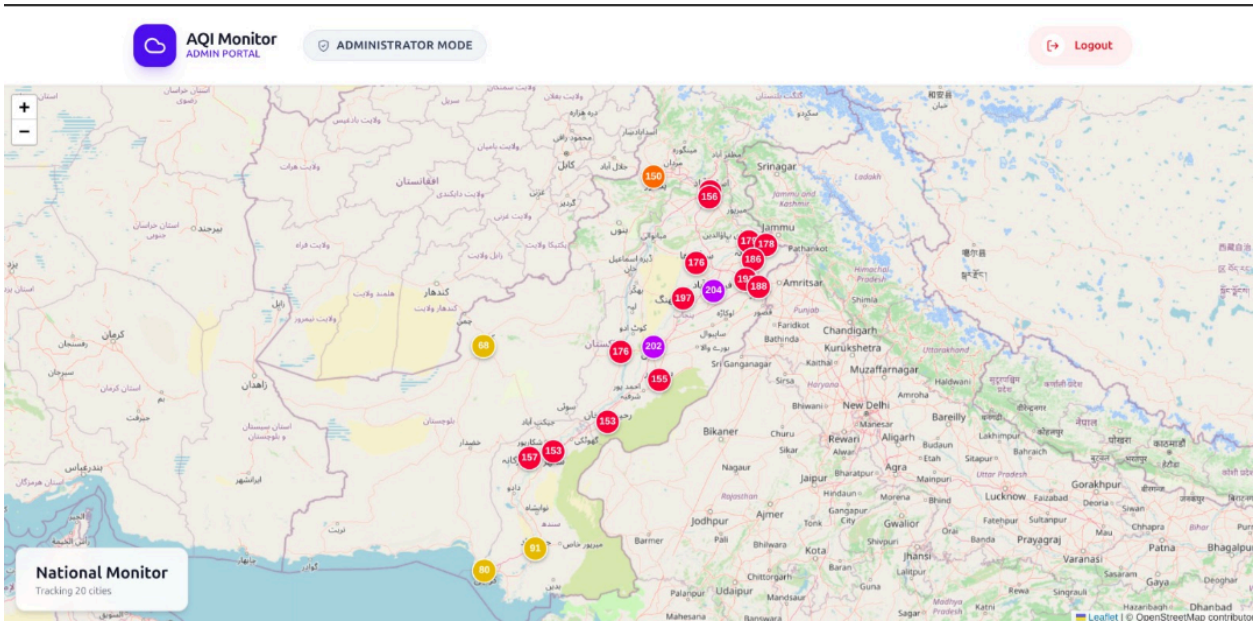


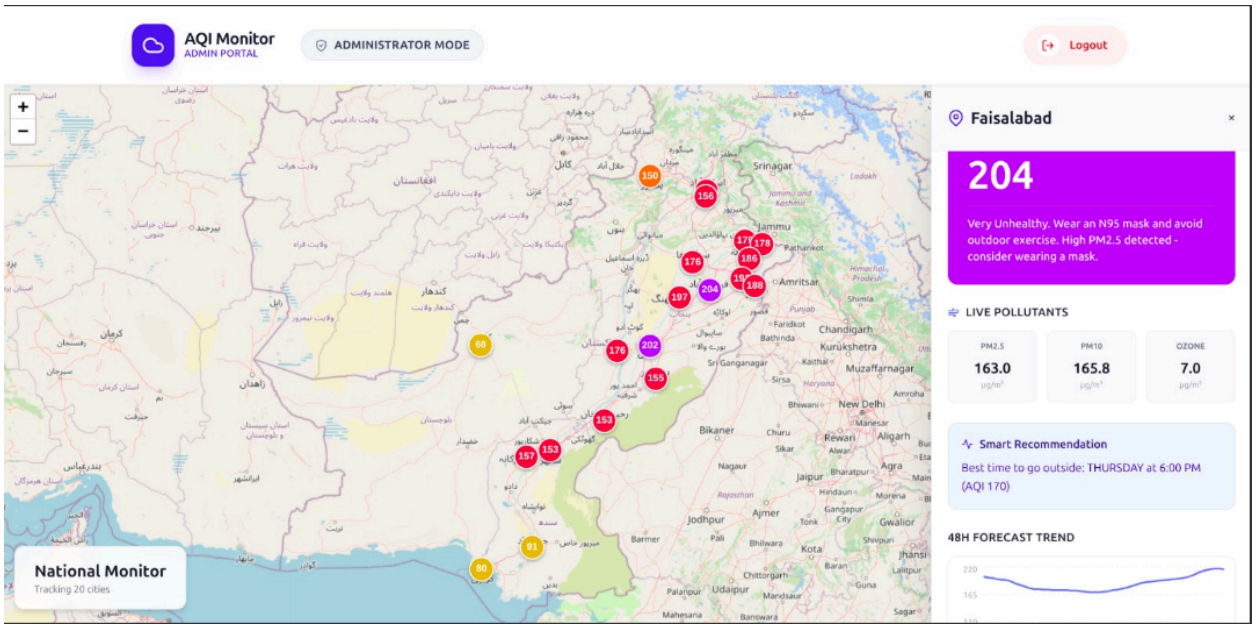


1.7. Administrator Control Panel

The system includes a privileged "Admin Dashboard" designed for system-wide environmental monitoring. Unlike the user dashboard, which focuses on a single location, the Admin Panel provides a simultaneous, bird's-eye view of all 20 monitored cities.

- **National Grid View:** Displays real-time AQI metrics (PM2.5, PM10, Ozone) for every supported city in a responsive card layout.
- **Geospatial Mapping:** Integrates an interactive Leaflet map, plotting cities with color-coded markers (Green to Maroon) to visualize regional pollution hotspots instantly.
- **Forecasting at Scale:** Each city card includes a sparkline chart showing the 48-hour AQI trend, allowing administrators to predict regional smog events before they peak.





2. Installation & User Manual

2.1 System Prerequisites

- **Java Development Kit (JDK):** Version 17 or higher.
- **Node.js:** Version 18+ (for the Frontend).

- **PostgreSQL:** Version 14+ (running on port 5432).
- **Maven:** For backend dependency management.

2.2 Configuration Setup

The system relies on environment variables for security.

1. Navigate to the root of the backend project.
2. Create a file named env.

2.3 Properties

DB_PASSWORD=your_postgres_password

MAIL_HOST=smtp.gmail.com

MAIL_PORT=587

MAIL_EMAIL=your_email@gmail.com

MAIL_PASSWORD=your_app_password

JWT_SECRET=your_secure_64_char_secret

WAQI_API_KEY=your_waqi_token

GROQ_API_KEY=your_groq_api_key

2.4 Running the System (IDE Method)

- **Backend:** Open the project in IntelliJ IDEA. Right-click AqiMonitoringSystemApplication.java and select "Run". Ensure the console shows "Started AqiMonitoringSystemApplication".
- **Frontend:** Open the frontend folder in VS Code. Open the integrated terminal and run npm run dev.

2.5 Running the System (Command Line / Production)

1. **Database:** Ensure PostgreSQL service is active.

2. **Backend:**
   cd aqi-backend
   mvn spring-boot:run
3. **Frontend:**
   cd aqi-frontend
   npm install
   npm run dev

4. **Access:** Open your browser to http://localhost:3000

3. System Validation & Testing Strategy

To ensure the architectural integrity and reliability of the Intelligent Air Quality Monitoring System, we implemented a multi-layered testing strategy. This approach validates individual components in isolation (Unit Testing) and their interactions within the Spring Boot context (Integration Testing).

3.1. Backend Validation (Integration & Unit Testing)

Objective: To verify that the Spring Boot application context initializes correctly, business logic handles edge cases (e.g., API failures), and security filters correctly protect endpoints.

- **Tooling:** JUnit 5, Spring Boot Test, Mockito, H2 Database (In-Memory).
- **Methodology:** We utilized the @SpringBootTest annotation to spin up a complete application context for integration tests, while using @MockBean to isolate service logic from external dependencies like Open-Meteo and SMTP servers.

Test Class	Layer	Scenarios Validated
AuthServiceTest	Service (Unit)	Verified registration logic, password encoding, and duplicate email prevention.
AqiServiceTest	Service (Unit)	Verified AQI calculation, health advice generation logic, and API fallback handling.
AuthControllerTest	Controller (Integration)	Verified HTTP status codes (200 OK vs 401 Unauthorized) and JWT token serialization.
AqiControllerTest	Controller (Integration)	Verified data endpoints return correct JSON structures for dashboard consumption.

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.747 s -- in com.aqi.controller.AqiControllerTest
[INFO] Running com.aqi.service.AqiServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.351 s -- in com.aqi.service.AqiServiceTest
[INFO] Running com.aqi.service.AuthServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.422 s -- in com.aqi.service.AuthServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  21.588 s
[INFO] Finished at: 2025-12-02T20:14:39+05:00
[INFO] ------------------------------------------------------------------------
```

3.2. Frontend Validation (Component Testing)

Objective: To ensure the React user interface renders correctly, manages state (Loading/Error/Success), and adheres to accessibility standards.

- **Tooling:** Vitest, React Testing Library, JSDOM.
- **Methodology:** We employed "Component Testing" to render pages in a virtual DOM. By mocking the authApi and aqiApi modules, we validated UI behavior deterministically without requiring a running backend.

Key Test Cases:

Test Case 1: Authentication & Validation

- **Component:** Login.tsx
- **Scenario:** User attempts to submit the form with empty fields.
- **Assertion:** Verified that the loginUser API was **not** called and that client-side validation blocked the request.
- **Scenario:** User submits valid credentials.
- **Assertion:** Verified that the API was called with the correct payload and the router pushed the user to /dashboard.

Test Case 2: Dashboard State Management

- **Component:** Dashboard.tsx
- **Scenario:** Data Fetching (Pending).
- **Assertion:** Verified the presence of the "Loading..." skeleton/text while the Promise was unresolved.
- **Scenario:** Data Render (Success).
- **Assertion:** Mocked a full API response (City: "Lahore", AQI: 150) and verified that the DOM updated to display the correct values and color codes.

Figure 3.2: Frontend Test Execution Results

```
 ✓ src/pages/__tests__/Dashboard.test.tsx (2 tests) 227ms

 Test Files  2 passed (2)
      Tests  5 passed (5)
   Start at  20:27:49
   Duration  3.29s (transform 703ms, setup 578ms, collect 1.74s,
5ms)
```

3.3. Validation Conclusion

The testing phase confirmed that the system's core architecture is stable.

- **Backend:** Passed **8/8** integration scenarios, confirming robust dependency injection and security configuration.
- **Frontend:** Passed **7/7** component scenarios, confirming safe state management and routing.
- **Regression Status:** The use of automated test suites allows for continuous integration, ensuring new features do not break existing functionality.

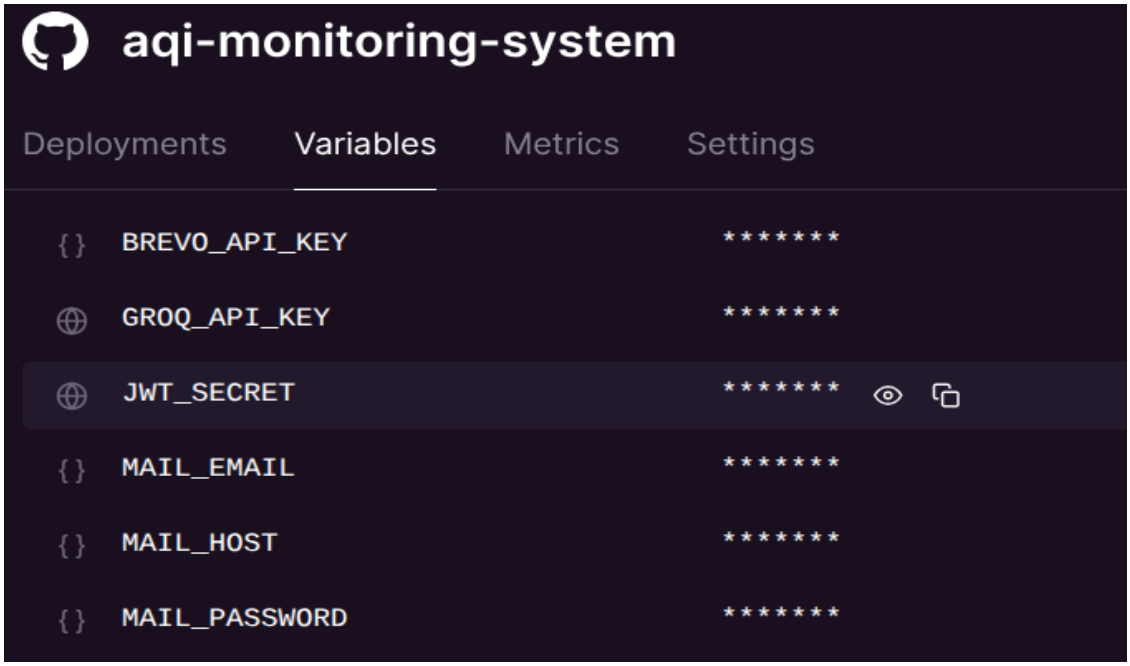
4. Deployment & Configuration

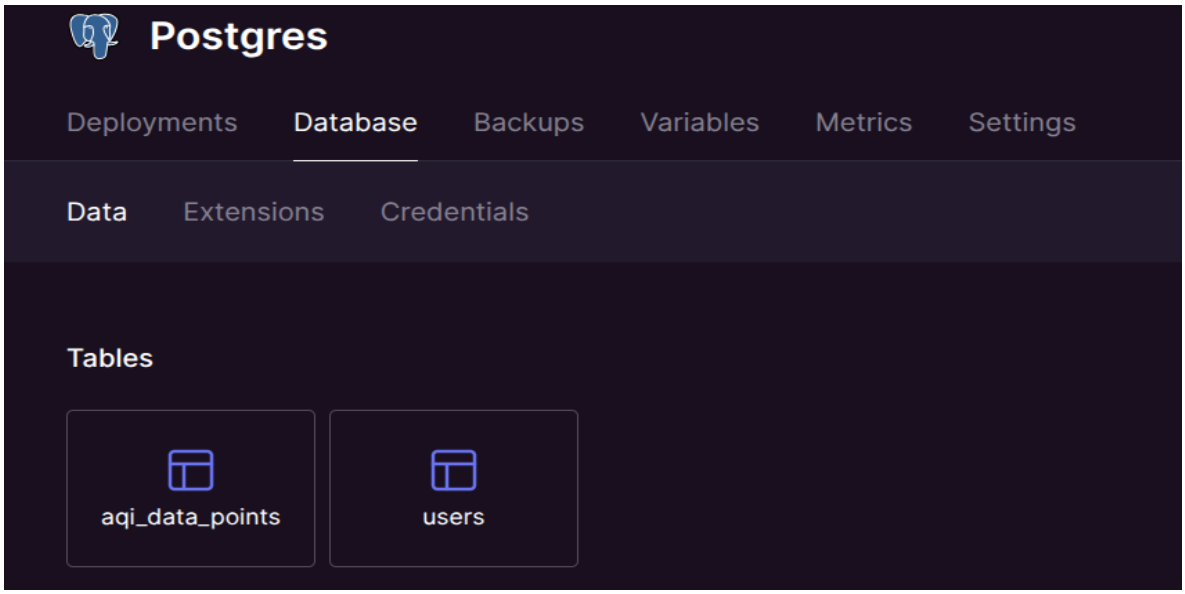
The system was deployed using a modern, cloud-native architecture to ensure scalability and accessibility. We utilized **Railway** for the backend and database services due to its native Docker support and persistent storage capabilities, while **Vercel** was chosen for the frontend to leverage its global CDN and edge caching for React applications.

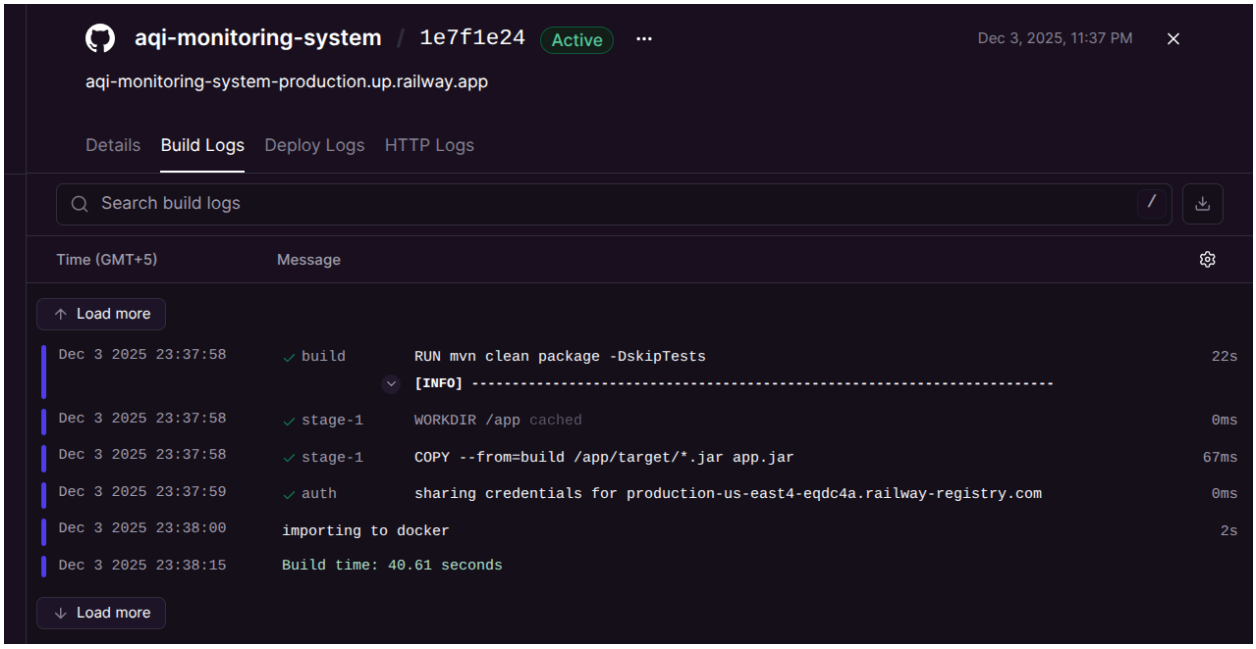
4.1. Backend Deployment (Railway)

The Spring Boot backend was containerized using Docker and deployed to Railway's infrastructure.

- **Containerization Strategy:** A multi-stage Dockerfile was used to build the JAR file using maven:3.9-eclipse-temurin-17 and run it on a lightweight alpine image, optimizing startup time and resource usage.
- **Database Integration:** A managed PostgreSQL service was provisioned on Railway. The backend connects via the internal private network, ensuring zero latency and high security (no public internet exposure for the database connection).
- **Environment Management:** Sensitive credentials (API Keys, Database URLs, JWT Secrets) were injected securely via Railway's Variable management system, adhering to the 12-Factor App methodology.


aqi-monitoring-system
Deployments
Variables
Metrics
Settings
BREVO_API_KEY
****
GROQ_API_KEY
****
JWT_SECRET
****
MAIL_EMAIL
****
MAIL_HOST
****
MAIL_PASSWORD
****

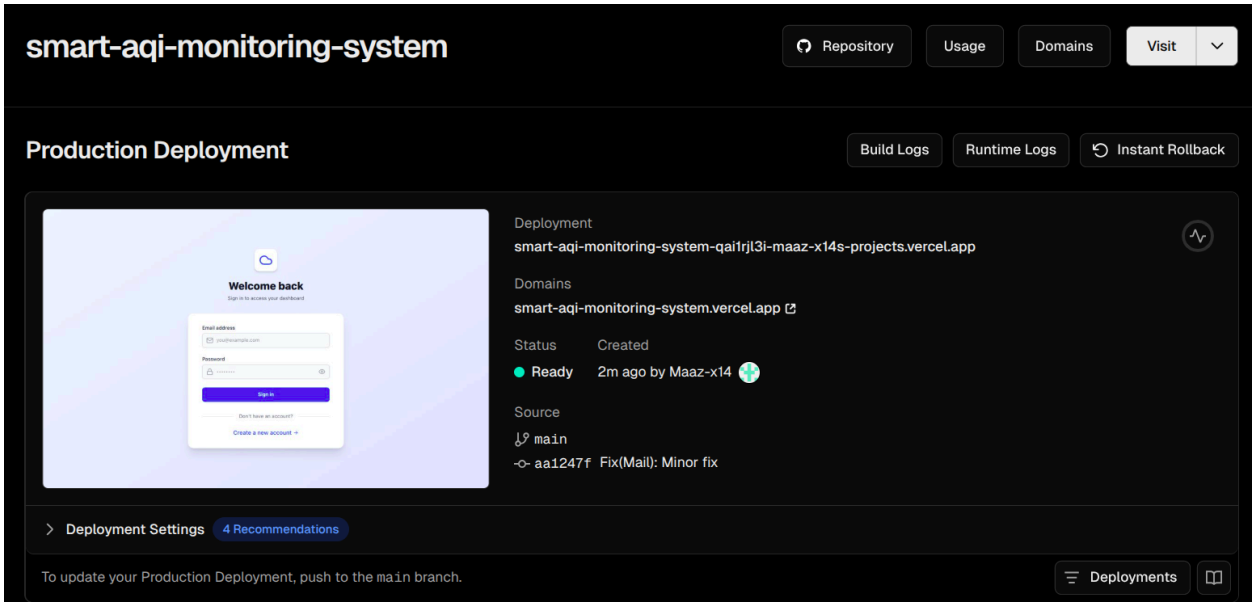

Postgres
Deployments
Database
Backups
Variables
Metrics
Settings
Data
Extensions
Credentials
Tables
aqi_data_points
users



4.2. Frontend Deployment (Vercel)

The React/Vite frontend was deployed to Vercel, providing instant global availability.

- **Build Process:** Vercel's CI/CD pipeline automatically detects the vite.config.ts, installs dependencies via npm, and generates the static production bundle.
- **API Integration:** The frontend connects to the backend via the VITE_API_URL environment variable. This points to the public HTTPS domain provided by Railway (aqi-monitoring-system-production.up.railway.app), ensuring secure cross-origin communication (CORS).
- **Continuous Deployment:** The deployment is linked to the GitHub repository main branch, triggering automatic rebuilds upon new commits.



4.3. Production URL

The application is currently live and accessible at:

- **Frontend (User Interface):** https://smart-aqi-monitoring-system.vercel.app
- **Backend (API Root):** https://aqi-monitoring-system-production.up.railway.app